

Physics Informed Deep Operator Networks for Neutron Transport

William Eivik Olsen*
Institute for Energy Technology
 (Dated: May 22, 2026)

An article usually includes an abstract, a concise summary of the work covered at length in the main body of the article.

I. INTRODUCTION

Global climate mitigation targets motivate the rapid expansion of low-carbon energy sources, and technological advances have renewed interest in nuclear energy as a reliable, compact, and dispatchable option. Small Modular Reactors (SMRs), typically up to 300 MW(e) per unit, promise factory fabrication and modular deployment [1]. Within this landscape, microreactors at up to roughly 20 MW(e) have emerged as particularly attractive for transportable, remote, and mobile applications, enabling reliable power in settings where conventional infrastructure is limited [2].

Maritime propulsion has become a prominent candidate for such deployment. Shipping is responsible for about three percent of global greenhouse gas emissions today and could rise substantially without decisive countermeasures. The NuProShip project [3], funded by the Research Council of Norway, surveyed more than eighty reactor concepts and identified three leading candidates for ship integration: a helium gas-cooled microreactor developed by Ultra Safe Nuclear Corporation, a molten-salt SMR by Kairos Power, and a lead-cooled SMR by Blykalla. While these technologies show promise, NuProShip highlights crew availability and cost—especially the need for highly trained nuclear operators—as a principal barrier to adoption. This challenge extends beyond maritime settings to the broader class of remote and mobile applications that motivate microreactors in the first place.

To mitigate this challenge, the development of digital twin technology for microreactors has been suggested [4]. A digital twin is a digital replica of an operating asset that can receive data from live sensors and provide physics modeling and simulation of the asset. In the nuclear context, a complete physics simulation should include the neutronics of the core, thermal-hydraulics and radiation transport [5]. An additional advantage of creating a digital twin is the ability to perform multi-physics simulations of the reactor in the approval phase. For this reason, digital twins for maritime nuclear propulsion has gained the attention of Det Norske Veritas (DNV), who recently initiated a research project to investigate model based safety assessments of maritime nuclear vessels using multi-physics simulations.

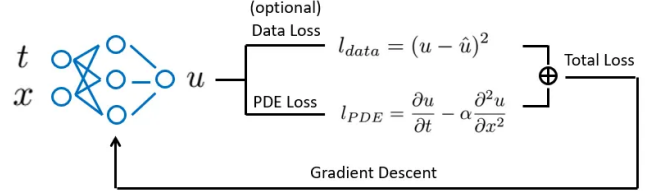


FIG. 1. Illustration of a PINN. Adapted from [11].

The heart of such a multi-physics model is the simulation of the reactor core itself – specifically, simulating the neutron transport and the fission chain reaction from which the core releases energy. The neutron transport equation (NTE) is notoriously difficult to solve. Traditional solution methods can be divided into *Deterministic methods*, in which simplifying assumptions are used to approximate the NTE, and *Monte Carlo methods*, in which the NTE is simulated using statistical methods. Both methods are able to provide solutions of satisfying accuracy. However, a digital twin should be able to simulate the reactor in real-time [6]. Deterministic and Monte Carlo methods both suffer from high computational costs, and are unable to solve the NTE in real-time [7–9].

The advance of machine learning methods has sparked interest in applying them to enable fast solutions of the NTE. Among them is Sahadath et al. [9], who investigated a DeepONet trained on neutron transport of a 1D slab. In this paper, we will investigate the performance of a *Physics Informed DeepONet*, first introduced by [10]. We will evaluate its performance on the same 1D problem and compare it to that of Sahadath et al.

II. MACHINE LEARNING METHODS

A. Physics-Informed Neural Networks

Prantikos et al. [12] developed a physics-informed neural network (PINN) to compute the neutronics with improved speed and accuracy. A Physics-Informed Neural Network (PINN) is a type of deep learning model designed to solve problems governed by physical laws, specifically those described by partial differential equations (PDEs) [13]. It uses a standard neural network to approximate the unknown solution as a function of input variables. The key idea is that the network is trained

* Also at Department of Physics, University of Oslo.

not only to match available data points, but also to satisfy the governing PDE itself within the domain, as illustrated in Figure 1. This constraint is incorporated into the training process by calculating the PDE residual using automatic differentiation and minimizing it alongside the standard loss function.

An advantage of PINNs in neutronics is their ability to perform fast computations. While the training process can require significant computational resources, once trained, the model can make predictions quickly. PINNs have been studied by several authors in the context of neutronics. Prantikos et al. [12] implemented a PINN model trained on the point kinetics equations and tested its predictions using experimental data from Purdue University Reactor Number One (PUR-1). More recently, studies have explored PINNs trained with the neutron diffusion equation [14] and the neutron transport equation [15].

One disadvantage with PINNs is that they require retraining for new input boundary/initial conditions [9]. This is challenging for the maritime settings, where inputs are expected to vary dynamically.

B. Deep Operator Networks

The disadvantage we discussed with PINNs are shared with most traditional machine learning methods, as they learn pointwise mapping between input-output data pairs. Deep operator networks, on the other hand, learn mappings between entire input and output function spaces. DeepONets treat inputs and outputs as functions, and aim to learn the solution operator that maps initial or boundary condition functions to corresponding solution functions [9]. This framework of learning the function mapping allows them to generalize to previously unseen conditions without the need for retraining. A DeepONet can therefore act as a sufficient surrogate model for the neutron transport equation, delivering accurate predictions for a wide range of scenarios while maintaining high generalization performance.

A DeepONet consists of two neural networks:

- A TrunkNet that processes evaluation points, such as time or spatial coordinates.
- A BranchNet that encodes discretized condition functions, such as initial and boundary conditions.

The final output is obtained through the dot product of the output of these two networks. An illustration of the DeepONet architecture is shown in figure 2.

Consider an input function $u(x)$ that can be sampled at a set of discrete positions $\{x_1, x_2, \dots, x_m\}$ in the domain D_x as $[u(x_1), u(x_2), \dots, u(x_m)]$. Given an operator G that acts on $u(x)$, $G(u)$ represents the resulting output function in domain D_y . Note that the domains D_x and D_y are not necessarily the same and can be entirely different. Within the domain of $G(u)$, the real value at any

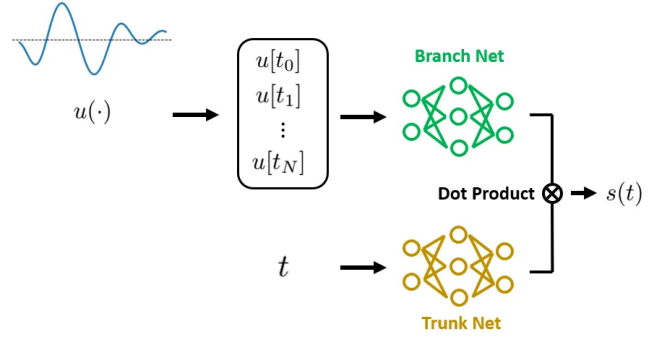


FIG. 2. Illustration of a Deep Operator Network. Adapted from [11].

sampling point y_i is expressed as $G(u)(y_i)$. From the generalized universal approximation theorem for operators, for any $\epsilon > 0$, there exists positive integers n, p and continuous vector functions $\mathbf{g} : \mathbb{R}^n \rightarrow \mathbb{R}^p$ and $\mathbf{f} : \mathbb{R}^d \rightarrow \mathbb{R}^p$ such that

$$|G(u)(y_i) - \langle g(u(x_1), \dots, u(x_m)), f(y_i) \rangle| < \epsilon, \quad (1)$$

where $\langle \cdot, \cdot \rangle$ denotes the dot product defined in $\mathbb{R}^p$, and the functions g and f can be represented by any neural networks.

The approximation of $g(u(x_1), \dots, u(x_m))$ is made by the BranchNet. This is done by mapping the input $[u(x_1), u(x_2), \dots, u(x_m)]$ to a vector $\mathbf{b}_N = [b_1, b_2, \dots, b_p]$. The TrunkNet takes an evaluation point y_i from the domain of the target operator $G(u)$ and outputs a vector $\mathbf{t}_N = [t_1(y_i), t_2(y_i), \dots, t_p(y_i)]$. Finally, the output is computed by

$$G(u)(y_i) \approx \sum_{k=1}^p b_k(u(x_1), \dots, u(x_m)) t_k(y_i). \quad (2)$$

The challenge faced by DeepONets is their requirement of large annotated datasets consisting of input-output observations. Another worry for our usecase is the model's ability to extrapolate: If the reactor enters a state significantly outside the training distribution, the DeepONet predictions can become unphysical.

C. Physics-Informed DeepONets

We have so far introduced PINNs and DeepONets. Both have strengths that make them suitable for solving differential equations in physics, but, as we have seen, each of them have their drawbacks:

- PINNs require retraining for every new initial and boundary condition. This is problematic for the maritime case, where we expect the conditions to vary dynamically.

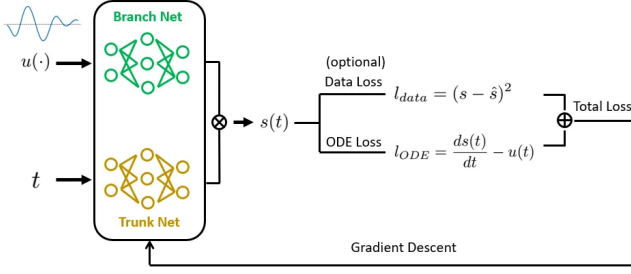


FIG. 3. Illustration of a Physics-Informed DeepONet. Adapted from [11].

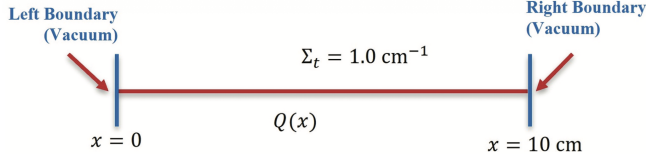


FIG. 4. Illustration of the 1D slab geometry. Adapted from [9].

- DeepONets require large annotated datasets of input-output observations, which for our case means running a large amount of Monte Carlo simulations of the reactor design we wish to model.

A promising method to circumvent this issue is to combine these two methods into a single framework: A Physics-Informed DeepONet. The architecture is visualized in figure 3. In addition to the Branch and Trunk net, the model incorporates the governing differential equation in its loss function. The architecture was first suggested by [10], who demonstrated its effectiveness on various types of PDEs, including a diffusion model.

The framework is able to combine the advantages of PINNs and DeepONets: They keep the low-data requirement of the PINN methods while being able to generalize like a DeepONet. The physics constraint makes the model more robust to handle inputs outside the training distribution.

For these reasons, we believe Physics-Informed DeepONets to be the most promising architecture for our use-case. The authors that originally suggested the framework have all their code available online [16]. This could serve as a starting point to develop a model of our own. Another excellent resource we found to learn about this is this blog post [11].

III. NEUTRON TRANSPORT PROBLEM

We adapt the problem from Sahadath et al. [9]. The geometry is a 1D slab with a length of 10 cm, spatially uniform cross sections and zero incoming current boundary conditions imposed at both ends. The geometry is illustrated in Figure 4. The total cross section is assumed

to be 1.0 cm^{-1} , while the absorption and scattering cross section depend on the scenario considered.

The steady-state one-group NTE for this 1D slab is given by

$$\begin{aligned} \mu \frac{\partial \psi(x, \mu)}{\partial x} + \Sigma_t \psi(x, \mu) &= \int_{-1}^1 \frac{1}{2} [\Sigma_{s0} + 3\mu\mu' \Sigma_{s1}] \psi(x, \mu') d\mu' \\ &+ \frac{Q(x)}{2}, \\ 0 < x < X, \quad -1 \leq \mu \leq 1 \\ \psi(0, \mu) &= 0, \quad 0 \leq \mu \leq 1 \\ \psi(X, \mu) &= 0, \quad -1 \leq \mu \leq 0 \end{aligned} \quad (3)$$

Here $\psi(x, \mu)$ is the angular flux at position x along the slab and direction cosine μ , where μ denotes the cosine of the angle between the neutron's direction of travel and the x -axis. The total macroscopic cross section is Σ_t , while Σ_{s0} and Σ_{s1} are the zeroth and first Legendre moments of the differential scattering cross section, respectively, with Σ_{s1} accounting for linearly anisotropic scattering. The external neutron source distribution is denoted $Q(x)$, and X is the slab length. The boundary conditions correspond to vacuum (zero incoming current) at both ends of the slab: no neutrons enter from the left at $x = 0$ for forward-traveling directions ($\mu > 0$), and none enter from the right at $x = X$ for backward-traveling directions ($\mu < 0$).

IV. DATA GENERATION FOR TRAINING

In the paper by Sahadath et al, a set of source distributions $Q(x)$ were prepared, and the 1D NTE was solved numerically for each of them to generate a training dataset for their DeepONet model. This was accomplished by first applying spatial and angular discretization. The spatial domain was partitioned into J uniformly spaced cells using the cell-averaged finite difference method. Denoting the edges of the j 'th cell by $x_{j \pm \frac{1}{2}}$, the cell width h_j and cell centre x_j are given by

$$h_j = x_{j+\frac{1}{2}} - x_{j-\frac{1}{2}}, \quad (4)$$

$$x_j = \frac{1}{2} (x_{j+\frac{1}{2}} + x_{j-\frac{1}{2}}). \quad (5)$$

Then we have the cell average p_j of a function $p(x)$ over the j 'th cell defined as

$$p_j = \frac{1}{h_j} \int_{x_{j-\frac{1}{2}}}^{x_{j+\frac{1}{2}}} p(x) dx. \quad (6)$$

The discrete ordinates (S_N) method is employed for angular discretization, using Gauss-Legendre quadrature with $N = 16$ angular bins. After discretizing space and

angle, the 1D NTE (3) can be rewritten to

$$\begin{aligned} & \frac{\mu_n}{h_j} \left(\psi_{n,j+\frac{1}{2}} - \psi_{n,j-\frac{1}{2}} \right) + \Sigma_{t,j} \psi_{n,j} \\ &= \frac{1}{2} (\Sigma_{s0,j} \varphi_{0,j} + 3\mu_n \Sigma_{s1,j} \varphi_{1,j} + Q_j), \\ & 1 \leq n \leq N, \quad 1 \leq j \leq J \\ & \psi_{n,\frac{1}{2}} = 0, \quad 1 \leq n \leq \frac{N}{2}, \quad \mu_n > 0 \\ & \psi_{n,J+\frac{1}{2}} = 0, \quad 1 \leq n \leq \frac{N}{2}, \quad \mu_n < 0. \end{aligned} \quad (7)$$

Here n and j are the angular bin and spatial cell indices, μ_n is the cosine of the scattering angle for direction n . The angular fluxes at the right and left cell edges are denoted $\psi_{n,j+\frac{1}{2}}$ and the cell-average total cross section is $\Sigma_{t,j}$, and $\Sigma_{s0,j}$ and $\Sigma_{s1,j}$ are the cell-average zeroth and first Legendre moments of the differential scattering cross section, respectively. The cell-average external source is Q_j and the cell-average scalar flux $\varphi_{0,j}$ and current $\varphi_{1,j}$ are obtained from the angular flux via Gauss-Legendre quadrature,

$$\varphi_{0,j} = \sum_{n=1}^N \psi_{n,j} w_n, \quad (8)$$

$$\varphi_{1,j} = \sum_{n=1}^N \psi_{n,j} \mu_n w_n, \quad (9)$$

where w_n are the Gauss-Legendre quadrature weights. The cell-edge and cell-average angular fluxes are related using the diamond difference approximation,

$$\psi_{n,j} = \frac{1}{2} \left(\psi_{n,j+\frac{1}{2}} + \psi_{n,j-\frac{1}{2}} \right). \quad (10)$$

The algebraic equation (7) can be solved by making an initial assumption of $\varphi_{0,j}$, $\varphi_{1,j}$ and $\psi_{n,j}$, then updating each of them iteratively according to (7) until convergence.

The spatially varying neutron source $Q(x)$ is modeled as a sample drawn from a Gaussian random field (GRF). A GRF is a stochastic process in which the value at each spatial point is a random variable, and the joint distribution across space is multivariate normal, fully specified by a mean function $m(x)$ and a covariance kernel $C(x_i, x_j)$. The mean function sets the expected value of $Q(x)$ — physically, the expected neutron emission rate density of the internal source — at each location x , while the covariance kernel encodes how the values of $Q(x)$ at different spatial points are correlated. Following Sahadath et al. [9], the covariance is given by the squared exponential kernel

$$C(x_i, x_j) = \sigma^2 \exp\left(-\frac{(x_i - x_j)^2}{2\ell^2}\right), \quad (11)$$

where σ^2 is the variance, controlling the amplitude of the fluctuations of $Q(x)$ about its mean, and $\ell > 0$ is the length scale, which governs the spatial smoothness of

the realizations — smaller values of ℓ correspond to more rapidly varying source profiles.

The kernel is evaluated at the midpoints of the $J = 100$ uniformly spaced spatial cells $x_1, x_2, \dots, x_J$, producing a $J \times J$ covariance matrix C . A single realization of $Q(x)$ is generated by sampling from the multivariate normal distribution defined by the mean vector $M = [m(x_1), m(x_2), \dots, m(x_J)]$ and the covariance matrix C . In practice, this is done by computing the Cholesky factorization $C = LL^\top$, drawing a standard normal vector $Z \sim \mathcal{N}(0, I)$, and forming

$$Q = M + LZ. \quad (12)$$

The result is a J -dimensional vector $Q = [Q(x_1), Q(x_2), \dots, Q(x_J)]$ giving the source value at each cell centre. Negative entries, which are unphysical for a neutron emission rate, are clipped to zero. Repeating this procedure with independent draws of Z yields the population of source samples used to train and test the DeepONet.

A total of 500 source distribution samples $Q(x)$ were generated with a mean of 5.0 and a length scale of 0.1, divided into 100 cells.

V. TRAINING

The models of Sahadath et al were set up and trained.

A. Model for Isotropic Scattering Media (M_{Iso}^{Sc})

The first model was developed under the assumption of only isotropic scattering in the slab geometry. The cross section parameters were set to $\Sigma_t = 1.0 \text{ cm}^{-1}$, $\Sigma_a = 0.5 \text{ cm}^{-1}$, $\Sigma_{s0} = 0.5 \text{ cm}^{-1}$ and $\Sigma_{s1} = 0 \text{ cm}^{-1}$.

Three variants of the Physics Informed DeepONet were trained:

- Data, physics and boundary condition loss.
- Data loss.
- Physics and boundary condition loss.

For comparison, an ordinary DeepONet using the same parameters as Sahadath et al. was also trained.

VI. RESULTS

A. Model for Isotropic Scattering Media (M_{Iso}^{Sc})

1. Benchmark

The benchmark loss is shown in Figure 5. One sample loss along with the true and predicted scalar flux are shown in Figure 6.

TABLE I. DeepONet performance on the M_{Iso} test scenarios from [9]. Values are reported as mean $\pm$ standard deviation across the 200 GRF samples per scenario; the LC, NLC, and sinusoidal cases use a single source realization each.

Scenario	n	R^2	ARE [%]
NS (no shift)	200	0.989 ± 0.005	1.1 ± 0.2
SS ₁ (small shift)	200	0.990 ± 0.006	1.1 ± 0.2
SS ₂ (small shift)	200	0.988 ± 0.006	1.2 ± 0.3
SS ₃ (small shift)	200	0.990 ± 0.005	1.1 ± 0.2
LS ₁ (large shift)	200	0.98 ± 0.01	1.6 ± 0.4
LS ₂ (large shift)	200	0.993 ± 0.004	0.9 ± 0.2
LS ₃ (large shift)	200	0.9962 ± 0.0003	0.76 ± 0.01
LS ₄ (large shift)	200	0.9962 ± 0.0003	0.761 ± 0.009
LS ₅ (large shift)	200	0.9963 ± 0.0003	0.760 ± 0.006
LS ₆ (large shift)	200	0.9963 ± 0.0004	0.758 ± 0.005
LC (linear comb.)	1	0.99308	0.847
NLC (nonlinear comb.)	1	0.97514	1.924
SIN _{LF} ($f = 0.2$)	1	0.99793	0.583
SIN _{HF} ($f = 0.8$)	1	0.99583	0.658

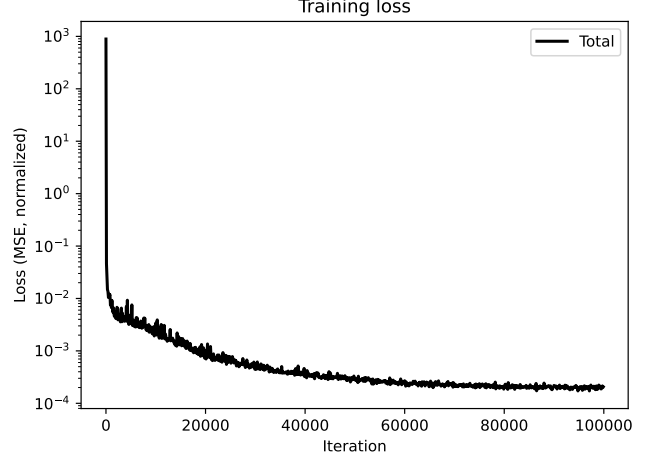


FIG. 5. Benchmark loss.

2. PI DeepONet, Data and Physics Loss

The data, physics and boundary condition loss are shown in Figure 7. One sample loss along with the true and predicted scalar flux are shown in Figure 8.

3. PI DeepONet, Physics Loss

4. PI DeepONet, Data Loss

VII. CONCLUSION

-
- [1] International Atomic Energy Agency, *Small Modular Reactors: Advances in SMR Developments 2024*, Tech. Rep. (International Atomic Energy Agency, Vienna, 2024).
 - [2] G. Black, D. Shropshire, K. Araújo, and A. van Heek, *Nuclear Technology* **209**, S1 (2023).
 - [3] J. Emblemavåg, C. Ordóñez, C. G. Tamm, J. S. Ortigosa, Y. M. Colilles, A. Pérez, H. Thoresen, E. Ovrum, M. J. F. Jordahl, L. Laanke, *et al.*, (2024).
 - [4] K. Wilsdon, J. Hansel, M. R. Kunz, and J. Browning, *Progress in Nuclear Energy* **163**, 104813 (2023).
 - [5] H. Mengyan, Z. Xueyan, P. Cuiting, Z. Yixuan, and Y. Jun, *Annals of Nuclear Energy* **202**, 110491 (2024).
 - [6] B. Kochunas and X. Huan, *Energies* **14**, 4235 (2021).
 - [7] P. Shen, X. Guo, K. Li, S. Huang, L. Zheng, Q. Pan, and K. Wang, *Annals of Nuclear Energy* **167**, 108861 (2022).
 - [8] F. Zhou, Y. Yang, X. Wu, P. Fang, Q. Liang, H. Lai, Y. Guo, Y. Zhu, and L. Yang, *Annals of Nuclear Energy* **208**, 110759 (2024).
 - [9] M. H. Sahadath, Q. Cheng, S. Pan, and W. Ji, *Nuclear Science and Engineering*, 1 (2026).
 - [10] S. Wang, H. Wang, and P. Perdikaris, *Science advances* **7**, eabi8605 (2021).
 - [11] S. Mousavi, “Operator learning via physics-informed DeepONet: Let’s implement it from scratch,” <https://medium.com/data-science/operator-learning-via-physics-informed-deeponet-lets-implement-it-from-scratch-1e1e1e1e1e1e> (2023), accessed: 2026-02-16.
 - [12] K. Prantikos, L. H. Tsoukalas, and A. Heifetz, *Energies* **15**, 7697 (2022).
 - [13] M. Raissi, P. Perdikaris, and G. E. Karniadakis, *Journal of Computational physics* **378**, 686 (2019).
 - [14] M. H. Elhareef and Z. Wu, *Nuclear Science and Engineering* **197**, 601 (2023).
 - [15] Y. Xie, Y. Wang, and Y. Ma, *Annals of Nuclear Energy* **195**, 110181 (2024).
 - [16] S. Mousavi, G. Pang, and G. E. Karniadakis, “Physics-informed DeepONets GitHub repository,” <https://github.com/PredictiveIntelligenceLab/Physics-informed-DeepONets> (2022), accessed: 2026-02-16.

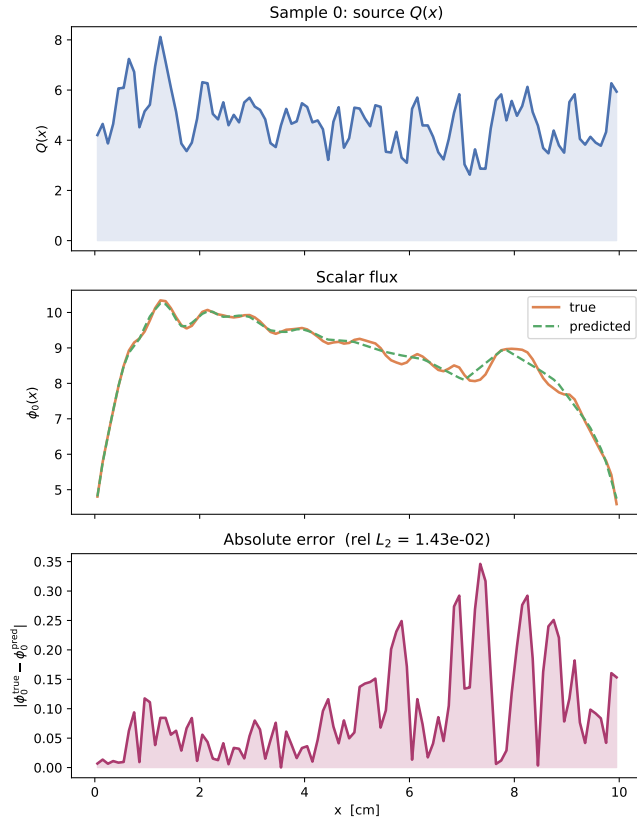


FIG. 6. Benchmark sample.

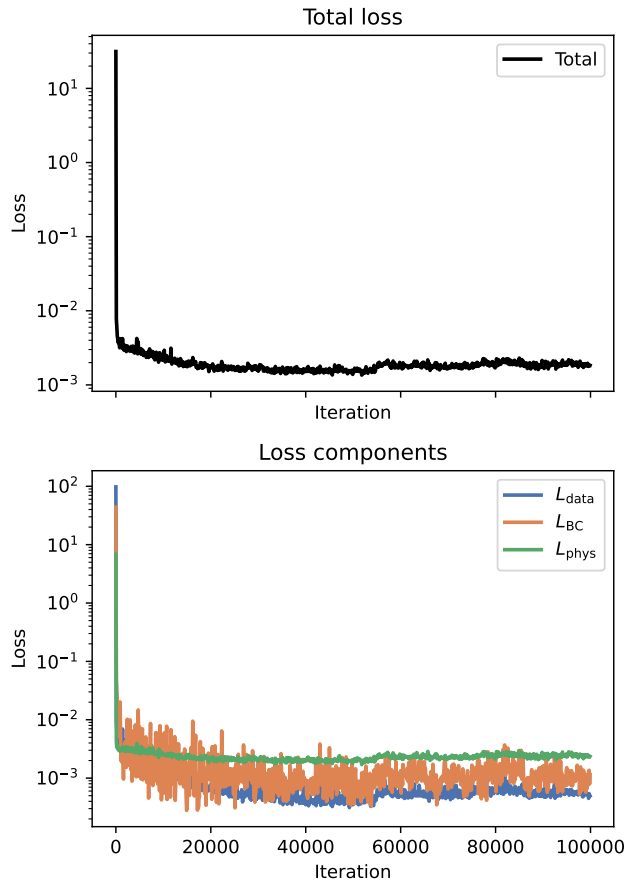


FIG. 7. Loss of data, physics and boundary condition trained model.

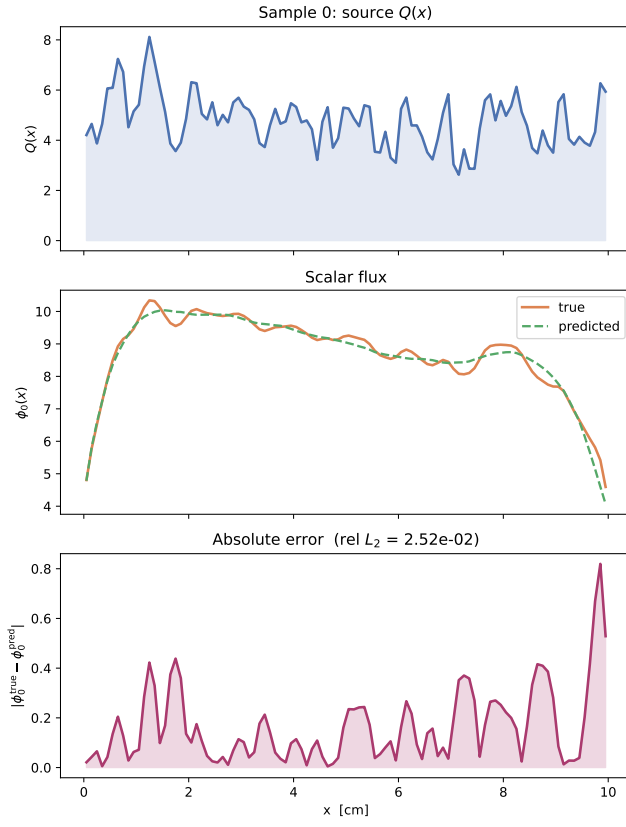


FIG. 8. Sample of data, physics and boundary condition trained model.